CHAPTER SIX

Polymorphism

Polymorphism is the ability of something to appear in multiple forms, depending on context, and the ability of different things to appear the same in a certain context. These are the two flip sides of the polymorphic coin. A refinement occurs once it is accepted that something does not have to be only what it appears to be. Then, dynamic exploration of what is behind the static appearance becomes possible.

This chapter sheds some light on the consequences of the above, somewhat philosophical, statements. In particular, the notion of substitutability is explained and related to the concepts of types and subtypes. Types lead to the notion of interfaces and their important role for components. As components are independently deployed, it is of utmost importance that the deployment environment accepts components that independently extend this environment. The important paradigm of independent extensibility is introduced to capture this requirement. The discussion of independent extensibility naturally leads to questions of security, safety, and trust. The chapter concludes with a discussion of different dimensions of independent extensibility, aspects of software evolution, and a short summary of some other forms of polymorphism not previously discussed in this chapter.

1.6 Substitutability – using one for another

Self-standing contractually specified interfaces decouple clients and providers. The same interface may be used by large numbers of different clients but also be supported by a large number of different providers. To avoid reducing the spectrum of possible clients and providers unnecessarily, an interface must be specified carefully. It should not require too much from either its clients or its providers. However, if it requires too little from them, the interface is just as useless.

If an interface requires too little from clients – for example, no non-trivial preconditions – it can overburden providers. In particular, such an interface can be hard or even impossible to implement. Also, it can be impossible to implement it

efficiently. On the flip side, if an interface requires too little from providers – for example, no non-trivial postconditions – it is of no use to clients.

A carefully crafted interface requires no more than is essential for the service to be provided. Such interfaces usually leave significant headroom for clients and providers. In particular, clients and providers are always free to overfulfill their contract. A client may establish more than is required by the precondition or expect less than is guaranteed by the postcondition. Likewise, a provider may require less than is guaranteed by the precondition or establish more than is required by the postcondition.

Pre- and postconditions are usually specified using predicates. The notion of expecting less than is guaranteed takes the form of a logical implication – guarantees imply expectations. Likewise, an implication can be used to express that more might be provided than is required – the provided implies the required. Using the common symbol for implication (⇒), the above can be summarized as:

established by certain client
⇒ demanded by interface (precondition)
⇒ required by certain provider

established by certain provider
⇒ guaranteed by interface (postcondition)
⇒ expected by certain client

Here is an example. Consider the interface TextModel as introduced in Chapter 5 (p. 67). The relevant section of the interface is repeated below.

```
interface TextModel {
  int max ();   // maximum length this text can have
  int length ();   // current length
  char read (int pos);   // character at position pos
  void write (int pos, char ch);   // insert character ch at position pos
    // [ len: int; txt: array of char •
    //   pre    len := this.length(); (all i: 0 ≤ i < len: txt[i] := this.read(i)) :
    //           len < this.max() and 0 ≤ pos ≤ len
    //   post  this.length() = len + 1
    //     and  (all i: 0 ≤ i < pos: this.read(i) = txt[i])
    //     and  this.read(pos) = ch
    //     and  (all i: pos < i < this.length(): this.read(i) = txt[i - 1])
    // ]
  ...
}
```

Notice how the interface requires callers of operation write to make sure that the character to be written is inserted within the current range of the text. An implementation GreatTextModel may relax this by allowing insertions to happen past

the end of the current text, by padding with blanks where necessary. Thus, a provider's implementation of write may have the following weakened precondition and strengthened postcondition (the changes are emphasized).

```
class GreatTextModel implements TextModel {
  ...
  void write (int pos, char ch) {
    // [ len: int; txt: array of char •
    //   pre     len := this.length(); (all i: 0 ≤ i < len: txt[i] := this.read(i));
    //           len < this.max()  and  0 ≤ pos < this.max()
    //   post  this.length() = max(len, pos) + 1
    //     and  (all i: 0 ≤ i < min(pos, len): this.read(i) = txt[i])
    //     and  this.read(pos) = ch
    //     and  (all i: pos < i ≤ len: this.read(i) = txt[i - 1])
    //     and  (all i: len < i < pos: this.read(i) = " ")
    // ]
    ...
  }
  ...
}
```

The precondition of the interface TextModel.write does indeed imply that of the implementation GreatTextModel.write:

```
  this.length() < this.max()  and  0 ≤ pos ≤ this.length()
⇒
  this.length() < this.max()  and  0 ≤ pos < this.max()
```

The postcondition of the implementation GreatTextModel.write does indeed imply that of the interface TextModel.write, provided the stronger precondition of the interface held. If it did not, the interface makes no statement at all about possible postconditions:

```
  ( from precondition:
    len := this.length(); max := this.max();
    (all i: 0 ≤ i < len: char[i] := this.read(i))
  )
  len < max  and  0 ≤ pos ≤ max
and
  this.length() = max(len, pos) + 1
  and  (all i: 0 ≤ i < min(pos, len): this.read(i) = char[i])
  and  this.read(pos) = ch
  and  (all i: pos < i ≤ len: this.read(i) = char[i - 1])
  and  (all i: len < i < pos: this.read(i) = " ")
⇒
```

```
this.length() = len + 1
and (all i: 0 ≤ i < pos: this.read(i) = char[i])
and this.read(pos) = ch
and (all i: pos < i < this.length(): this.read(i) = char[i - 1])
```

In both cases, the verification of these implications is straightforward and left as an exercise to the reader so inclined. Readers preferring not to go into this level of detail may safely skip this exercise.

There are, surprisingly, many further possible ways in which providers can interpret the interface contract. For example, a provider may accept arbitrary positions, including negative ones, in all operations, simply by first clipping the specified position to the currently valid range. Alternatively, a provider may decide to grow a text dynamically, making max return a non-constant value. Equally, a provider may decide to create the illusion of an infinitely long text, preinitialized with all blanks.

Obviously, clients based on the interface contract will not be able to benefit from any of the above implementation refinements. However, the same implementations could also support another interface contract that reveals more general capabilities.

On the client's end of an interface, the same sort of relaxation is possible. A client may guarantee more than is required and expect less than is provided. For example, a client of TextModel.write may use the operation only to append to a text:

```
{ TextModel text; int pos; char ch;
  ...
  if text.length() < text.max() {
    text.write(text.length(), ch);
    // expect  text.read(text.length() - 1) = ch
  }
  ...
}
```

Notice how pre- and postconditions make it fairly difficult to express conditions such as: "if text.length() < text.max() at one time, then this will continue to hold for as long as text.length() does not change." Such a condition is vitally important to avoid a "race condition" with precondition checks – the example above, for instance, depends on it. Owing to its notions of abstract time and sequencing, a history or statement-based specification technique as introduced in Chapter 5 can naturally cover such detail.

As is again easily verified, the client-established condition, together with the TextModel.length postcondition, implies the TextModel.write precondition. Also, the TextModel.write postcondition, together with the client's guarantees, implies the client's expectation:

```
  text.length() ≥ 0
and
  text.length() < text.max()  and  pos = text.length()
⇒
  text.length() < text.max()  and  0 ≤ pos ≤ text.length()
  ( from precondition:
     len := this.length(); max := this.max();
     (all i: 0 ≤ i < len: char[i] := this.read(i))
  )
  pos = len
and
  text.length() = len + 1
  and  (all i: 0 ≤ i < pos: text.read(i) = char[i])
  and  text.read(pos) = ch
  and  (all i: pos < i < text.length(): text.read(i) = char[i - 1])
⇒
  text.read(text.length() -1) = ch
```

Understanding the flexibility introduced by requiring only implications, instead of equivalences, becomes very important when considering interactions of multiple providers and clients. Figure 6.1 illustrates the pivotal role of the interface contract when considering multiple clients and multiple providers of the services advertised by an interface.

Libraries always supported the concept of a service provider catering for many clients. However, in any one given configuration, there is only one implementation of a library interface and the only concern on the provider's side is versioning. As explained above, the situation has changed dramatically with the introduction of self-standing interfaces and dynamic dispatch (late binding).

When is it legal to substitute one service provider for another? An unknown number of clients may rely on the service simply by relying on what is contractually promised by the service interface. Therefore, another service provider can come in if it satisfies the same contract. If a provider satisfies the same contract as another, the former is said to be substitutable for the latter.

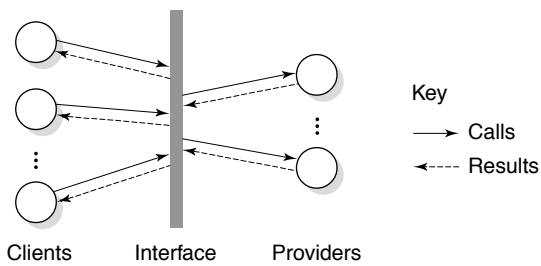


Figure 6.1 Pivotal role of interface contracts, coupling many clients with many providers.

6.2 Types, subtypes, and type checking

Ideally, all conditions of a contract would be stated explicitly and formally as part of an interface specification. In addition, it would be highly desirable to have a compiler or other automatic tool check clients and providers against the contract and reject incorrect ones. In practice, this ideal is quite far from attainable. Fully formalizing the interface contracts is a first major obstacle. Doing so is difficult and often considered too expensive.

However, even where a formal contract is available, automatic checking remains a major challenge of ongoing research. It is known that an efficient general-purpose verifier is not feasible. Therefore, research concentrates on tools that use heuristics to automate most of the manual proof process. Such tools are called theorem provers. Theorem proving will be too expensive to be done in a production compiler for years to come, and it requires manual assistance by an expert.

Today's compilers cannot check most conditions in a contract. Some cannot even be checked efficiently at runtime. However, it is worthwhile aiming for the earliest possible error checking to rule out potentially catastrophic faults. Hence, compiler-time checking is better than load-time checking, load-time checking is better than runtime checking, and runtime checking is better than no checking. However, sometimes later checking is the only possibility. For example, version conflicts among independently provided components can only be checked at configuration or load time. Likewise, index range errors can, generally, only be detected at runtime.

A class of errors that can lead to particularly disastrous results is memory errors. A memory error occurs when a program reads a memory cell based on a wrong assumption of what the cell contains. The program may expect, say, an object reference, whereas the cell really contains either something different – for example, a floating-point number – or has not been initialized at all. Memory errors are among the worst because they can affect entirely unrelated parts of a program, are notoriously hard to track down, and can be arbitrarily destructive.

Fortunately, there is a way to deal automatically with those conditions that need to be verified to eliminate memory errors. The idea is to group all values of related semantics, such as all integers, into sets. Such sets are called types. A type system ensures that all memory accesses are type compatible. By combining a type system with automatic memory management and certain runtime checks, a language implementation can fully eliminate memory errors – and some other error classes as well. Many modern languages, including Smalltalk, Java, C#, Oberon, and Component Pascal, fall into this category. However, many others, including Object Pascal, Modula-2, C, and C++, do not.

Basic types, such as INTEGER or REAL, can be understood as sets of values. In the context of objects and interfaces, a type is best understood as a set of objects implementing a certain interface.

A type names all operations of an interface and the number and type of parameters and the type of returned values of each of the operations. The types of input and in–out parameters form a part of an operation's preconditions. The types of output and in–out parameters plus the type of returned values form a part of an operation's postconditions. However, other pre- and postconditions that are likely to be part of the underlying contract are not part of a type. Thus, a type is an interface with a simplified contract.

As a type is a simplified contract, it is possible that a program that passes type checking still violates some contracts. This is an important point and is discussed below in section 6.5.

An object may implement more than the interface required in a certain context. It may implement additional interfaces or a version of the required interface extended with additional operations or both. For instance, the interfaces View, TextView, and GraphicsView below are in such an extension relation. TextView and GraphicsView are both interfaces that implement all operations in View, but also add operations specific to text and graphics viewing respectively. A more complete set of TextView operations has already been presented in Chapter 5 (p. 69).

```
interface View {
  void close ();
  void restore (int left, int top, int right, int bottom);
}
interface TextView extends View {
  int caretPos ();
  void setCaretPos (int pos);
}
interface GraphicsView extends View {
  int cursorX ();
  int cursorY ();
  void setCursorXY (int x, int y);
}
```

Under what circumstances can an object implementing one type be used in a context expecting another type? The answer follows directly from the above discussion of contracts. A client expecting a certain type really expects an object fulfilling a contract. Thus, all objects fulfilling the contract could be used. On the level of types, this includes all extended types as long as the understanding is that objects of an extended type respect the base contract.

Obviously, a TextView object or a GraphicsView object can be used wherever a View object is expected. As the set of text views is a proper subset of the set of views, TextView is called a subtype of View. Likewise, GraphicsView is a subtype of View.

The view examples show the formation of a subtype by defining a new interface and naming a base interface that is to be extended by the new interface.

This is called interface inheritance and is the most common way to form subtypes. Another way is structural subtyping, in which no base interface is named. Instead, operations are repeated and a subtype is said to be formed if a subset of the operations coincides with operations defined for another type.

A variable of type View can refer to objects of type View, TextView, GraphicsView, or other subtypes. This is called polymorphism, and the variable is said to be of a polymorphic type. As there are several different forms of polymorphism, this particular one is also called subtype or inclusion polymorphism. The latter name refers to the inclusion of subsets in their base set.

As far as the type system is concerned, objects of a subtype are substitutable for objects of a base type. However, again a warning: just as types simplify contracts, so does subtype polymorphism simplify substitutability. The fact that an object is in a proper subtype relationship does not guarantee that the object's implementation respects the base contract, or, for that matter, any contract. On the other hand, it is fair to say that types and subtyping rules can statically prevent many of the outright "dangerous" violations of contracts.

6.3 More on subtypes

The types of the parameters and the return value of an interface's operations form part of the pre- and postconditions of that operation. Discussion on "overfulfilling a contract" by a service provider helps to understand what legal modifications to the types of an operation a subtype interface may apply. (It would be logical to talk about subcontracts when referring to the contracts associated with subtypes. Unfortunately, subcontract has an entirely different and well-established meaning.)

Note that this section mostly presents straightforward consequences of what has been discussed above, but is quite technical in nature. It can be safely skipped, especially on a first read.

Types of output parameters and return values form part of an operation's postconditions. A provider may establish more than is required by a contract. Hence, a subtype interface can replace the types of output parameters and return values by something more specific – that is, by subtypes. In other words, the types of output parameters and return values may be varied from types to subtypes when moving from an interface of a certain type to an interface of a subtype of that type. As the types of output parameters and return values can thus be varied in the same direction as the types of the containing interfaces, this is called covariance.

Types of input parameters form part of an operation's preconditions. A provider may expect less than is guaranteed by a contract. Hence, a subtype interface could replace the types of input parameters by something more general – supertypes. In other words, the types of input parameters may be varied from types to supertypes when going from an interface of a certain type to an interface of a subtype of that type. As the types of input parameters can thus

be varied in the opposite direction of the types of the containing interfaces, this is called contravariance.

The requirements of co- and contravariance can be illustrated graphically by showing how a function could be substituted for another if it covers the same or a larger domain and if it has the same or a smaller range. Figure 6.2 illustrates this. In terms of domain and range, function g in the figure could be used in places where function f was expected.

Finally, types of in–out parameters simultaneously form part of an operation's pre- and postcondition. From a combination of the arguments for types in pre- and postconditions developed above, it follows that types of in–out parameters cannot be varied at all in a subtype interface. This phenomenon is sometimes called invariance of in–out parameter types. If, in addition to operations, an interface is also allowed to contain modifiable attributes, these also have invariant types.

Here is an example. Consider adding an operation getModel to interface View. In the case of a text view the model is of type TextModel, and this could be reflected by covariantly redefining getModel in TextView. The same can be done for getModel in GraphicsView.

```
interface View {
  ... // as above
  Model getModel ();
}
interface TextView extends View {
  ... // as above
  TextModel getModel ();
}
interface GraphicsView extends View {
  ... // as above
  GraphicsModel getModel ();
}
```

This is obviously useful. Clients that care only about View will get a generic Model when they ask for the views model. However, clients that know they are dealing with a TextView object will get a TextModel.

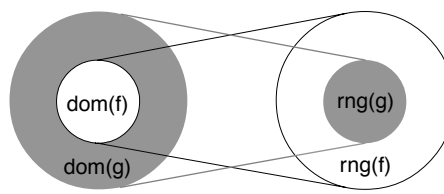


Figure 6.2 Contravariance of the domain and covariance of the range of a function *g* that could, in terms of domain and range, be substituted for a function *f*.

Next, consider an operation setModel used to connect a view to a certain model. If part of View, setModel would have to take an object of type Model.

```
interface View {
  ... // as above
  void setModel (Model m);  // is this a good idea?
}
```

However, a TextView object needs a TextModel object as its model and a GraphicsView needs a GraphicsModel. If covariant change of input parameters was safe, setModel could be changed in TextView to accept a TextModel and in GraphicsView to accept a GraphicsModel.

Using a subtype interface allows substitution of a subtype object for a base type object. Thus, a text view would have to expect to be used in a context that knows only about interface View. Within that context, the precondition of setModel simply requires, as far as types are concerned, a Model object. Thus, even if TextView were allowed (incorrectly) to modify covariantly the type of setModels input parameter, it would not help. The best that could happen is a dynamically caught type violation error.

Some type systems allow the introduction of typings that are coupled to the interface inheritance hierarchy. In other words, part of the precondition of the base operation is that subtype operations may covariantly modify certain parts that normally would fall under the contravariance restriction. Such type systems have to introduce other restrictions to remain sound, that is, to continue to allow claims about absence of certain errors in well-typed programs.

For example, for static checking, the original Eiffel type system (Meyer, 1990) required a conservative global analysis of entire programs including all libraries used. Such a global analysis was called system-level type checking, and it clearly defeats the ideas of modular checking (p. 77). A library may type check on its own and thus be delivered to clients. Then, the combination of client and library code may lead to type errors in the library. Such type errors are indirectly caused by the addition of the client code, but are hard to explain. Modular checking requires monotonicity, which means that if one module is determined to be internally type correct, then the addition of other modules cannot affect this result.

Type matching (Bruce *et al.*, 1997) and the revised Eiffel type system (Meyer, 1996) both allow for modular type checking of individual operations but require a more specific typing of variables. In particular, type matching requires that covariant changes in contravariant positions can occur only where the typed variable is declared to be monomorphic – that is, cannot refer to an object of a subtype. Similarly, revised Eiffel requires that such covariant changes can only occur where the typed variable can be conservatively determined to be monomorphic. Sather is another object-oriented language that requires variables to be declared as either monomorphic or polymorphic (Szyperski *et al.*, 1994; Omohundro and Stoutamire, 1996).

6.4 Object languages and types

Some languages, such as Smalltalk, do not have an explicit type system. In some of these cases, a compiler can still derive types by inspection of strictly local program fragments, such as in the Smalltalk dialect StrongTalk (Bracha and Griswold, 1993). In such languages, avoidance of explicit typing is a matter of convenience – there is less to write. The compiler can still check a program's typing and static type safety is preserved. Adding explicit typing is still useful, as it makes important architectural and design decisions explicit.

In other cases, including original Smalltalk, type inference is not possible without relying on global analysis of the entire code body (Palsberg and Schwartzbach, 1991). Type checking in Smalltalk and similar languages is thus deferred to runtime. For example, a Smalltalk expression `x insert: 4 at: 3` requires that variable `x` refers at runtime to an object that understands the message `insert:at:`. However, in the general case, this cannot be verified statically. As a result, `x` may refer at runtime to an object that does not understand the message `insert:at:`. At least, this is checked dynamically and a "message not understood" exception is raised at runtime in such a case.

More modern languages, such as Java and Component Pascal, use an explicit type system and statically check programs at compile time. In addition, they check narrowing type casts at runtime. (A narrowing cast is a cast in which the programmer asserts that a variable of a certain type really refers to an object of a subtype of that type. The cast narrows a set to a subset.)

For readers who followed the co-/contravariance story above, it is interesting that few of the mainstream languages support any changes in types of operations when forming subtypes. In C++, covariant return types were introduced only in early 1994 (Ellis and Stroustrup, 1994, p. 421). Java and C# still do not support any type changes. (The Java 1.0 beta specification allowed for covariant return values, while neither the alpha specification nor the final 1.0 or 1.1 specification does.) Like C++, Component Pascal supports covariant change of return values, but no other co- or contravariant changes.

6.5 Types, interfaces, and components

Some programming languages do not require the programmer to specify types and, instead, infer the most general types that still allow a program fragment to be type correct. Of course, this approach is impossible where interfaces are self-standing. At the time of creation of such an interface, there may be no implementations available. Indeed, clients may be programmed before the first provider implementing the interface becomes available. To keep clients and providers independent, a self-standing interface has to be fully and explicitly typed to benefit from type checking.

Other programming languages establish subtype compatibility between two types structurally. Assume that an interface of one type happens to contain all

operations contained by another type – and the operations in the two interfaces themselves are appropriately typed. In this case, the former type is inferred to be a subtype of the latter. As, in this case, subtyping is based on type structures, this is sometimes called structural subtyping in contrast to declared subtyping. Obviously, in situations in which all types are inferred, types do not have names and subtyping has to be based on structural compatibility.

When viewing types as simplified contracts, it becomes clear that structural subtyping is dangerous. The structure of a type covers only part of a contract – the part that can be expressed in the used type system. A named type refers to the full contract. If another type is then considered to be a subtype of this type, it has to respect the contract of its base type, in addition to its own contract. As the full contracts are only referred to, proper subtype relations cannot be inferred automatically. Thus, a programmer should explicitly declare whether or not a type should be considered a legal subtype of another type. If so declared, the programmer accepts the obligation to verify that the subtype relation is indeed justified.

Many articles (for example, Magnusson, 1991) describe the apocryphal tale of a graphics editor that accepted all objects that happened to have a method draw. Unfortunately, the user program failed horribly after accidentally inserting a cowboy object into the graphics editor, which then caused the editor to redraw all objects.

It is sometimes argued that accidental structural subtyping is not likely in practice. The argument is that, for this to happen, all operations of the base type, including all of their parameter numbers and types and their return types, accidentally need to agree. For substantial interfaces this is indeed unlikely. Indeed, even for the draw example above, it is not likely that a graphics objects draw method would do without any parameters – as is likely to be the case for a cowboy object's draw method.

For small interfaces, and these are typical at the roots of subtype hierarchies, the situation is different. Quite often, root types have no or only a few very basic operations. Additionally, root types tend to be designed by following common design patterns. Thus, it is indeed possible and likely that accidentally derived subtypes lead to objects in positions where they, unknowingly, break their contracts.

For example, base types for interfaces such as Event and Property are likely to be empty. Alternatively, they may contain an operation to get some "owner" object – the event source or the object having the property. Thus, the structure of base types, say, Event and Property may coincide. Deriving a subtype relationship from that is unacceptable, of course, as an event is not a property and vice versa.

Another aspect of components is their potential support of multiple interfaces. It is sometimes necessary to specify a required set of interfaces and then accept any component that provides at least these required interfaces. Such interface sets are sometimes called categories. For example, Microsoft's COM

IDL has been extended to support such categories. This can be used to support one-stop tests, such as checking that a component implements all interfaces required to make it an ActiveX control.

In Java and C# (and most other CLR-targeting languages), even objects can implement multiple interfaces and interfaces can extend multiple interfaces. An example would be the merger of the two TextView examples presented above (pp. 69 and 89) – a TextView interface would have to extend both the TextObserver and the View interface. In Java, this is possible. However, Java has no concept of categories and an object's implementation of a set of interfaces must therefore be tested one by one. However, since category-like sets of interfaces are known ahead of time, they can be expressed as empty interfaces that derive from the set of interfaces in question. Testing whether or not an object implements a particular category then amounts to checking if it implements the category's interface. This approach works only if the object in question has been equipped to actually implement the category's interface rather than just implementing all the individual interfaces required by the category. Using a notion of structural interface sets and checking for the matching of such sets solves this problem. An example is Büchi and Weck's compound types (1998).

6.6 The paradigm of independent extensibility

The principal function of component orientations is to support independent extensibility.

A system is independently extensible if it is extensible and if independently developed extensions can be combined (Szyperski, 1996). For example, all operating systems are one-level independently extensible by loading applications. However, many applications themselves turned into independently extensible systems. Extensions to applications are sometimes called plugins. Applications, such as Netscape's Navigator or Internet Explorer, slowly turn into platforms by supporting an increasing number of increasingly complex plugins. The first examples of plugins supporting "sub-plugins" have already arrived. One such example is Gazelle, a browser plugin that supports downloading of Component Pascal "applets" that execute in a local blackbox component framework (Paznesh, 1997). Java is by now part of many browsers, but where that is not the case, Sun's Java Activator plugin is another example.

As applications are fragmented and turned into extensible architectures, operating systems do not stand still either. A radical design reduces the operating system itself to a minimal kernel, called a micro-kernel, and farms out almost all OS functionality to application-level servers. Micro-kernels were originally driven by research operating systems such as Mach (Accetta *et al.*, 1986) that by themselves never really made it on to the more general markets. However, the micro-kernel design also influenced industrial-strength operating system designs, including Microsoft's Windows NT (Cutler, 1993), which led

to Windows 2000 and Windows XP, and recently Apple's Mac OS X (www.apple.com/macosx/), which is directly based on Mach.

Combining the "dekernelization" efforts of operating system architects and the modularization efforts of application architects leads to a new vision for overall system architectures – components everywhere! However, it is not only about being able to construct the equivalent of the traditional operating system combined with a handful of applications. It is all about forming a system architecture that is independently extensible on all levels. It is about independent extensibility as a recursive construction principle applied uniformly. Such systems have been explored in research projects (for example, the Ethos system; Szyperski, 1992a) and also in industrial projects. One of the most ambitious approaches, Taligent, failed. To understand why, it is helpful to return first to the early attempts to develop micro-kernel designs for operating systems.

Partitioning of systems into smallest components (to maximize reuse) conflicts with efficiency, but also with robustness when facing evolution and configurational variety. This was discussed in detail in Chapter 4. An argument similar to that for robustness can be made for performance. Micro-kernel architecture enforces total isolation of application-level processes to establish system safety and support security mechanisms. However, isolation comes at a price, and frequent crossing of protection domain boundaries can severely affect performance. (There are memory management units that separate address space management from protection domains, for example the ARM MMU (Furber, 1996). However, hardware support for many lightweight protection domains has yet to make it into the mainstream. Software alternatives exist (Wahbe *et al.,* 1993), but have yet to gain the degree of trust that a hardware protection mechanism commands.) In the end, for an operating system to be viable, its architects have to strive for a balance between flexibility on the one hand and performance and robustness on the other.

After initial waves of euphoria, many operating system experts now agree that extreme "micro-kernelism" is not the optimal design for an operating system. After all, flexibility and configurability are only one side of an operating system's characteristics. The other side, and at least as important, is delivery of overall system performance. It is the latter requirement that directly conflicts with extreme micro-kernel designs. For example, starting with version 4.0, Microsoft moved significant parts of the display driver code into the NT kernel to improve substantially the performance of graphics-intense applications. This move, unfortunately, also introduced a major threat to the robustness of NT – faulty display drivers would literally render polygons across kernel memory. Starting with Windows 2000, Microsoft is now publishing driver test suites and encourages driver manufacturers to get their drivers certified by independent third-party services. The certification authority cryptographically signs certified drivers – unsigned drivers of unknown quality status are, by default, rejected by Windows (as of Windows XP).

How can component technology and independent extensibility as a recursive system design concept ever be viable if performance is so severely affected? The answer to this question would seem extremely important for the future of component technology. In reality, however, it turns out to be the wrong question.

The true question to be answered is "Why is performance so severely affected?" The obvious answer is "because it is expensive to perform cross-context calls." Whenever an invocation crosses contexts (processes and so on), the operating system has to ensure several things. It has to make sure that security policies are respected – that is, check whether or not the call happens under proper authorization. Further, it has to adjust the hardware protection mechanisms by switching from caller to callee context. Finally, it has to make sure that the call parameters are transferred from the caller to the callee context. Once the called function returns, the process has to be reverted. In addition to the unavoidable basic costs, the switch also affects the processor caches. On some machines, the caches even have to be flushed for security reasons.

All this is expensive – a cross-context call on well-tuned operating systems is still easily a hundred times more expensive than a local in-process call. This is not the operating system's fault. Most architectures rely on hardware-supported process isolation to guarantee safety. An example of this is protecting a process in the presence of another out-of-control process that is trying to write to arbitrary memory addresses. Given a properly "sealed" operating system and proper set-up of authorizations, even a maliciously ill-formed program written directly in the machine's assembly language cannot harm other programs executing on the same machine.

The cost of hardware protection is high but can be tolerated if the switching can be limited to, for example, a few hundred switches per second. This is the case under normal time-sharing operations of traditional operating systems. In this case, inter-process communication (IPC) is usually based on pipelining data streams that decouple source and destination and reduce the frequency of context switching. Clearly, for tightly interacting components, "intercomponent communication" happens at much higher frequencies and is much more synchronous – the source component often has to wait for the result before it can continue.

On machines built for interactive use and made from cheap commodity items, called personal computers, this has long led to the utilization of rather different operating systems. Neither the Mac OS nor MS-DOS had a true process model. Hardware protection was mostly ignored. A malicious program could easily "crash" the entire system. This was, and occasionally still is, considered acceptable as these machines typically serve a single user who is in charge and who can avoid the crash simply by avoiding the use of unreliable applications.

At the same time, these machines and operating systems perform very well when it comes to typical interactive use. System extensions or plugins are called directly, with no context switches involved. Component architectures in

specialized areas have been in mainstream use for some time. For example, Apple's QuickTime included plugin modules from the start, even though QuickTime's multimedia services are among the most performance-critical a system can offer.

Is it possible to have your cake and eat it too? Could there be a third way, with efficient component composition that also offers the sort of safety already demanded in section 2.2? One way is to choose carefully the granularity of components. If most interactions stay within a component's boundaries, the cost incurred when crossing component boundaries may be tolerable. Chapter 8 continues a discussion of various aspects governing component granularity. Another way is to guarantee statically that a component will be safe. This is discussed further in the next section.

6.7 Safety by construction – viability of components

The discussion on type safety points in the right direction. After all, hardware protection "just" eliminates memory errors – it prevents one part of a system from accessing (reading, writing, or executing) any other part of the system for which it has no authorization. In a totally type-safe system, the equivalent reading is that no part can access any other part to which it has not been given a reference or that is not in its static scope of visibility.

Here is an example. The Java class files (Java's portable compiled format) and the Java virtual machine have been crafted to interact in a way that prevents type-unsafe applets from being executed in a non-local environment. Java is a type-safe language. It provides automatic memory management using garbage collection. Finally, it performs runtime checks on all operations that are "dangerous," but cannot be statically checked, such as array bounds checks. Together, these techniques guarantee that memory errors cannot occur. As class files, produced by the Java compiler, could be tampered with, the virtual machine rechecks them when loading one coming from a non-local site, such as across the internet.

Component Pascal is another example. The language offers similarly strong guarantees. Here the execution environment receives an intermediate portable form produced by the compiler that is based on an entirely different approach. This form is also rechecked, but then compiled into a local cache of binaries and future references to the same component directly use the cached binary. Like Java, Component Pascal is also fully garbage collected. Unlike Java, Component Pascal neither has nor needs an interpreter or a virtual machine.

A third example is the Microsoft .NET Common Language Runtime (CLR). CLR also uses late compilation and avoids interpretation, eliminating the notion of a virtual machine. It defines an intermediate language that has been designed to support a wide variety of programming languages and still map efficiently to a wide variety of processors. Some 15 to 20 languages have been investigated and many of them implemented (in a large international

collaborative effort called "Project 7") in order to validate the claim of support for many languages.

All three examples share one feature, which is that strong safety properties can be established at compile-time, checked at install, load, or JIT compile-time, and then be relied on without further checking while executing the resulting efficient code.

6.7.1 Module safety

However, type safety and elimination of memory errors are not enough, although it is assumed that the language respects object encapsulation. Without any additional measures, it would still be possible for a program to call arbitrary services present in or loadable into the system. This would be all that is required to acquire references to arbitrary objects in the system and thus "legally" (as far as the type system is concerned) to perform arbitrary manipulations. For example, there are ActiveX controls that shut down Windows.

The one additional requirement, also met by Java, Component Pascal, and CLR is module safety (Szyperski and Gough, 1995). A component has to specify explicitly which services – from the system or other components – it needs to access. This is done in the form of module (or package) import lists. The language and system do not allow access to any non-imported module. Hence, if that list contains only permissible modules, then it is safe to load and execute the new component. This is like access control in file systems, but on a per module (or per class) basis, rather than a per object basis. Objects are too dynamic to be identified explicitly for purposes of static access control.

With module safety in place, abstractions of entire multiobject services can be formed. Unlike classes, modules can enforce invariants across tightly interacting objects (Szyperski, 1992a). In a language not supporting any higher granularity of packaging than classes, access protection would have to be per class, offering class safety instead of module safety.

Module safety is not quite as simple as it sounds. In component systems it is important that other components (and services) can be retrieved by name. The name itself may be made available to a component after that component has been compiled, checked, and loaded – that is, at runtime. A clean and popular way to support component retrieval by name is a reflection service. Java, Component Pascal, and .NET CLR all offer such services. Where access to components is to be restricted, reflection services require special attention so as not to create a security loophole.

6.7.2 Module safety and metaprogramming

It should be impossible for a component to retrieve references to other components to which it has not been granted access. For example, in Java, a special security manager object is used to inspect whether or not the caller of a critical

function has proper authorization. To do so, the call stack is traversed and checked for any untrusted activation record. If there is none, the call can proceed as it cannot serve any untrusted component. The call to the security manager is performed by the critical operation itself to protect itself against unwanted calls. The security manager throws a security exception if it does not authorize access for the direct or one of the indirect callers.

As admitted in the JavaBeans standard document (Sun, 1996), this approach can be undermined by a carelessly programmed but trusted service that performs asynchronous requests on behalf of clients (a so-called event adapter). In such a case, the true client will not be active at the time of the safety-critical call, only the trusted service will be. Another loophole would be if a critical object were registered in an unprotected registry. In this case, anyone guessing the access key or name or, if it were supported, anyone enumerating all registered objects, could retrieve the critical object under a generic supertype, such as Object. It would then be simply a matter of type test and cast to regain access to the critical object.

Even if a component legally accesses some module, it must not gain any access to the private (non-exported) parts of that module. For direct access, this is guaranteed by the language semantics. However, where meta-programming interfaces exist, these need to be explicitly restricted such that these services do not break encapsulation. This is contrary to some of the typical usages of meta-programming, such as debugging or data structure serialization services. Indeed, a system may offer two metaprogramming interfaces – one that is module safe and open for general use and another that is module unsafe and restricted to trusted components.

The BlackBox Component Framework (used with Component Pascal) is currently a unique example of a system offering a dual metaprogramming service. The type- and module-safe service is open for general use, whereas the module-unsafe service is used by trusted components, such as the BlackBox portable debugger. The Java JDK includes metaprogramming facilities that are normally module safe (package safe). Some limited module-unsafe operations are available to trusted callers, subject to security manager validation.

An interesting twist is added by the .NET CLR. This is the possibility of adding custom meta-attributes to methods that restrict the set of callers based on scope (calling code from a specific manufacturer or from a specific other component only) or security demands (calling code has to provide acceptable credentials). Several of these demands are checked at load-time, some can only be checked at runtime. The load-time checks are particularly interesting and unique.

6.7.3 Safety in a multilanguage environment

Component technology allows the selection of different programming languages for the implementation of different components. Mutual protection of components in the same hardware protection domain thus requires type and

module safety properties that are common to all languages used in such a configuration. Sufficiently strong interface definition languages (IDLs) can solve this problem. It is therefore unnecessary to restrict a system to a single programming language approach simply to benefit from language-level type and module safety. However, the strength of the overall approach will depend on the strength of its weakest part – that is, the weakest of the languages used, including the IDL. The CLR provisions for establishing type safety, module safety, and security integrity across all supported languages show that the choice of language is second to the establishment of underlying firm principles compatible with many languages. Note, however, that requiring strong CLR safety properties limits the used languages to their safe subsets. For instance, C# supports unsafe methods, which are those that cannot be automatically checked for their safety. A component written using C#'s unsafe methods (or similar facilities in other CLR-supporting languages) will be rejected by the automatic safety checks unless that component is equipped with the credentials that declare it as trustworthy. Unlike the ActiveX Authenticode approach, where components are either fully trusted or not executed at all, and like Java components, CLR components are preferably usable without requiring such levels of trust.

6.8 Safety, security, trust

Type safety, module safety, and absence of memory errors – what makes this language–semantics-based approach trustworthy? The answer depends on the circumstances. If the target is a set of components, installed locally, interacting on a personal computer, then this approach may already be close to satisfactory. If the target was the highest security level, then this approach would be totally unacceptable. If the target is a low- to medium-security system that moves components across the internet, this approach is probably at its limits.

What is wrong? First, and above all, this approach fully relies on the tight semantics of the programming language used. To be fully trustworthy, a formal semantics of the language together with formal proofs of the claimed safety properties would be required. Also, of course, the formal method itself needs to be trusted, including the tools that may be used to construct lengthy proofs semiautomatically. Very few programming languages satisfy this requirement and there is no mainstream object-oriented language.

Even if a language fully satisfies this criterion, its implementation could still break any proved property. Hence, to go all the way, the language processors and the language runtime systems again need to be formally specified and verified. This is quite a feat to achieve. Consider that language processing includes tools such as a compiler, an interpreter, or a byte code verifier. The language runtime includes mechanisms such as a virtual machine, a garbage collector, or a security manager. Obviously, this may lead to a bootstrap problem where the tools used to build the used tools again need to be formally

verified, and so on. None of the widely available language implementations satisfies these criteria, not even in cases where the implemented language itself satisfies its safety criterion.

In the end, trust is a matter of reducing the unknown to the known and trusted, and doing so in a trusted way. This is obviously primarily a sociological process. For example, the way the Unix security mechanism was introduced played a major role in it being trusted. The designers of the mechanism published its details in full and encouraged everyone to try to break it. After years, the mechanism gained (or, better, earned) the trust it currently receives. This trust can be justified probabilistically. It is highly unlikely that a loophole remains undiscovered under the uncoordinated (stochastic) evaluation by many over an extended period of time.

The Java designers also publicized their security strategies early on and encouraged serious research groups (McGraw and Felten, 1997) to challenge the approach. In this way, known loopholes are published and fixed (see "Frequently asked questions – Java security," www.java.sun.com/sfaq). After years of steadily decreasing reports of found problems, people will increasingly trust the approach. Today, limitation of trust is appropriate, whatever the proponents claim, and even if they are right.

6.9 Dimensions of independent extensibility

In traditional class frameworks, extension happens by virtue of specialization: classes of the framework are subclassed to add the required behavior. If the framework does not offer the right abstractions, such a specialization may not be possible. However, the notion of implementation inheritance, as interpreted in most object-oriented languages, allows for quite radical changes to the classes introduced by the framework. Quite often, the required functionality can be forced on the framework.

Traditional class frameworks are specialized at application construction time and thereafter disappear as no longer separable parts of the generated application. The only conflict resulting from "forced" specialization is that migration of the application to the next release of the independently evolved framework may be difficult or impossible.

The situation is very different for independently extensible systems. Using a single component framework, extensions from independent sources can be combined. Forcing specialization would endanger the interoperability of independent extensions. Thus, even more so than for class frameworks, independently extensible systems require a clear statement of what can be extended. Each particular feature of an independently extensible system that is open for separate extension is called a dimension of (independent) extensibility (for example, Weck, 1997).

Note that dimensions of extensibility are not necessarily orthogonal – the same effect may be achieved by extending along one of several possible dimen-

sions. This is not normally desirable, as it offers designers of extensions an unwanted choice. Perfect orthogonality of dimensions is difficult to achieve and a total exclusion of possible overlaps may excessively restrict individual dimensions. The result would be a system that is orthogonal but not complete. As a result, important extensions become impossible. The theoretical ideal would be to form orthogonal dimensions of independent extensibility that together form an extension space that is complete with respect to extensibility requirements.

In practice, extensible systems rarely have orthogonal dimensions of extensibility. For example, the same system may support extensible abstractions for object serialization and object persistence. Although not identical, these two services certainly overlap. Multiple dimensions of extensibility form a product space (the set of all combinations of extensions along the individual dimensions). Where the dimensions are orthogonal, the resulting space is a Cartesian product. (A Cartesian product, also called cross-product, of n sets is the set of n-tuples containing all possible permutations of the elements of the n sets. The number of effectively different permutations is reduced if the original sets overlap and then the corresponding dimensions are non-orthogonal.)

6.9.1 Bottleneck interfaces

If a system was independently extensible "from scratch" – that is, no fixed infrastructure would exist, then independent extensions could not interoperate at all. There would not be any common ground for them to interact. A common ground after the fact could not generally be provided as the extensions come from independent and mutually unaware sources. Component frameworks provide the required shared understanding that couples extensions. A component framework opens a number of dimensions for extending components. Also, a component framework may enforce some of the rules of interactions between extensions.

Interfaces introduced to allow the interoperation between independently extended abstractions are sometimes called bottleneck interfaces (for example, Szyperski, 1992b). A bottleneck interface is a self-standing contract. As a bottleneck interface couples independent extension families, it cannot itself be extended. This is one of the arguments behind the claim that such an interface, once published, can only be withdrawn or replaced but not extended. Bottleneck interfaces, once published, are immutable (see also the next section). Obviously, components may be mutually aware of their extensions and engage in special interaction. In poorly designed component frameworks, such mutual awareness is sometimes required for two components to interact properly. In other words, such component frameworks require seemingly independent components to enter into a "conspiracy."

6.9.2 Singleton configurations

A component framework may open dimensions of extensibility but require a singleton configuration for some of the dimensions. A configuration is a singleton configuration, regarding a specific compulsory dimension, if it provides exactly one component that extends that dimension. For optional dimensions, a singleton configuration provides at most one component. For example, a system may allow for the installation of a security manager, but may insist on having at most one such manager in a configuration at any one time.

6.9.3 Parallel, orthogonal, and recursive extensions

Normally, for most or even all of its dimensions, a component framework would not require singleton configuration. Thus, a configuration can contain many components that all extend along the same dimension. This is called parallel extension (Weck, 1997). The main problem of systems allowing for parallel extension is peaceful coexistence. Two components extending along the same dimension are in danger of asking for the same resources. A component framework allowing for parallel extension has to define rules and provide means for arbitration. An example is a set of multiple controls embedded into the same container of a compound document system. Necessarily unique resources, such as the current focus for keyboard input, need to be arbitrated when requests from multiple controls collide.

Separate extending components may also address orthogonal dimensions of a component framework. This is called orthogonal extension (Weck, 1997). The main problem with orthogonal extension is the provision of proper bottleneck interfaces to allow interaction between the orthogonal extensions. Again, using the example of controls and containers in a compound document, controls and containers are addressing orthogonal dimensions. A single component may address both dimensions simultaneously, so, for example, it could be control and container at the same time. A component framework for compound documents has to define and support the bottleneck interfaces that allow arbitrary controls to talk to arbitrary containers and vice versa. A typical interface of this kind is used by controls and containers to negotiate for screen space.

Both parallel and orthogonal extensions are flat – that is, extending components are mutually independent. Recursive extensions are also possible. A component can itself introduce a component framework. Components extending this new framework obviously depend on the framework-introducing component or at least the abstractions introduced by this component. Note that recursive extensions do not necessarily create a well-layered architecture. It is possible that a component simultaneously extends dimensions introduced by recursive extensions at different levels. For example, a system might have a component framework for distributed resource management. One of the components extending this framework may itself be a framework for compound

documents. It is possible that, say, a container component extends dimensions of both the resource management and the compound document framework.

6.10 Evolution versus immutability of interfaces and contracts

A contract – an interface together with its specification – mediates between independently evolving clients and providers of the services the interface makes accessible. As soon as a contract has been published to the world, it (the interface and its specification) can no longer be changed. This holds for clients and providers bound by a specific contract.

A provider can always stop providing a particular interface. It will then potentially lose part of its client base – the part that has not yet been migrated to some newer interface. However, a provider can never change the specification of an existing contract as that would break clients without any obvious indication. Also, a client cannot change its understanding of the contract without risking breaking with some existing providers.

The cornerstone of even arguing about contracts and changing contracts is a way to name uniquely the contracts that clients and providers refer to. As a contract, with all its informal bylaws, is itself quite difficult to capture, the name of the associated interface is commonly used.

6.10.1 Syntactic versus semantic contract changes

Changes to a contract can take two forms. Either the interface or the specification is changed. If the interface is changed, this is referred to as a syntactic change. If the specification is changed, this is called a semantic change. As providers are often "in charge" of a contract, and typical providers in object-oriented settings are classes, the problem caused by contract change is sometimes referred to as the fragile base class problem. The syntactic and semantic variations of the fragile base class problem are discussed in detail in section 7.4.

A simple way to avoid these problems is to refrain from changing contracts once they have been published. Of course, there is no problem with changing contracts (syntactically or semantically), as long as all bound providers and clients are under control. Thus, evolution of contracts within a tight organization is usually not an issue. However, by releasing clients or providers to the open market, the contracts involved become uncontrollable. Then, change has to stop and the contracts have to be frozen, made immutable.

It is again helpful to consider the analogy of traditional contracts. No clause in a contract, once signed, can be changed without the agreement of all involved parties. Of course, once there is an uncontrollable number of parties, gaining agreement can become difficult or impossible. Examples are contracts between all employers and employees covered by some tariff union. However, such contracts do have mechanisms that allow for change. The two fundamental mechanisms are: acknowledging existence of overriding law and instances

and statement of a termination time. (Thanks to Wolfgang Weck for pointing out this analogy.)

It is noteworthy that, today, only Microsoft's COM declares all published interfaces to be immutable. Even Microsoft itself adheres to the principle and introduces new interfaces instead of modifying existing ones that almost fit. Support for the older interfaces is upheld for a while to maintain backwards compatibility, but eventually such older interfaces can and should go. IBM's SOM did something different. By explicitly supporting a release order, clients of different releases (versions) of an interface can coexist. Essentially, the explicit release order guarantees for every method a fixed index into a lookup table. For this to work, a new release can only add to an interface, it cannot take functionality away. A more detailed discussion of the two approaches can be found in Chapter 13 (SOM) and Chapter 15 (COM). SOM has since been deprecated. Ironically, many newer approaches discussed in this book, including J2EE's EJB and CORBA 3's CCM, follow the SOM approach of supporting release-to-release binary compatibility models. The .NET CLR supports both – the COM approach of side-by-side existence of interface (and class), versions that therefore can be kept immutable, as well as the SOM approach of release-to-release compatibility.

6.10.2 Contract expiry

Some of the current component infrastructures offer licensing services and it is a natural property of licenses that they expire after a preset date. Hence, it would seem straightforward to couple service provision under certain contracts with license agreements and their expiry date. The result would allow for a controlled evolution in which clients and providers can blindly trust the validity and existence of a service under a certain contract until the expiration date is reached. Thereafter, manufacturers of clients and providers would have to renegotiate: a contract's lifetime could be extended or the contract could be replaced or refined.

Using contracts with "use-by dates" has effects on users. Software systems can no longer be used indefinitely – or at least for as long as that outdated PC can be kept alive. However, there is also an advantage. Instead of supporting legacy contracts forever, adding more and more baggage to providers and clients, there is a clean way to cut off the past. As noted above, it is always possible to stop support for an aged contract. However, without mutually agreed expiry dates, this will always come as a surprise to some users.

6.10.3 Overriding law

What about the other mechanisms in traditional contracts, overriding law? Overriding law in this sense can mean the depreciation of clients or providers conforming to the old interpretation of a contract. In acts of self-justice,

strong companies or organizations often apply this principle. A more moral way to achieve the same is interception by an accepted independent organization. Typical examples are the International Standards Organization (ISO), the American National Standards Institute (ANSI), the European Computer Manufacturers Association (ECMA), the British Standards Institution (BSI), the German institute for industrial standards (DIN), and so on.

6.11 Other forms of polymorphism

There are other forms of polymorphism besides the subtype (or inclusion) polymorphism introduced above (section 6.2). These other forms are discussed briefly below. For thorough coverage, see Abadi and Cardelli (1996).

Although subtype polymorphism is a dynamic scheme, the other forms are static and resolvable by a compiler. There are higher-order type systems, where some of these other forms may also require dynamic resolution. Such type systems have not yet reached mainstream languages. Overloading, as supported by languages such as C++, Java, and C#, is a form of polymorphism that groups otherwise unrelated operations under the same name. Overloading is sometimes called ad hoc polymorphism because it establishes only a superficial similarity that is based on neither typing nor shared implementation.

The third form of polymorphism focuses on using the same implementation to serve a variety of types. For example, a list implementation can be parametrized with the type of the list elements. Any one instance of the list will serve one specific type, but the list implementation itself is generic and provided only once. This is called parametric polymorphism and is similar to generics in Ada. Properly implemented, parametric polymorphism does not lead to an explosion of generated code. A particularly lightweight variant that always produces exactly one copy of code has been proposed for Oberon (Roe and Szyperski, 1997) and may in the future be supported by Component Pascal. Note that parametric polymorphism is also similar to C++ templates. However, C++ templates lead to code explosion as a template is necessarily compiled to different code for each instantiation. Also, templates cannot be statically type-checked as type checking cannot occur before parameters are supplied.

It is sometimes useful to specify constraints on type parameters. Parametric polymorphism does not allow this – any type can be used to parameterize a parametric abstraction. For example, a parametric list can be used to form lists over arbitrary types. For lists, that is fine. Consider a parametric container component. It accepts controls of a certain type, determined by the type parameter. However, whatever the type parameter will be, it has to be a subtype of Control, that is, only control types are acceptable.

A stronger polymorphic form that accepts bounds in the form of minimal supertypes is called bounded polymorphism. Bounded polymorphism combines subtype and parametric polymorphism. For example, using subtype polymorphism, a list of control objects can be constructed, but the same list

implementation cannot be further constrained to contain only text controls. Using parametric polymorphism, the latter can be achieved, but the list could also be parametrized to contain only models (or any other type of objects) – the subtyping property that each element is at least a control is lost.

In the control container example, control-specific operations need to be applied to all list elements. For such operations to be statically safe, a combination of subtype and parametric polymorphism would be required and bounded polymorphism could be used. The list would be parametrized with a type that has to be a subtype of the control type. The control type places a bound on acceptable parameter types. An interesting challenge for the late integration into existing languages is the necessity to coexist with existing code that does not know about bounded polymorphism. A further challenge is the desire to fully support runtime reflection of type information even via variables of bounded polymorphic type. Proposals to integrate bounded polymorphism into Java (Myers *et al.*, 1997; Odersky and Wadler, 1997; Bracha *et al.*, 1998; Cartwright and Steele Jr, 1998; JCP, 2001) and into CLR and C# (Sime and Kennedy, 2001) exist and most of the challenges are now addressed in practical ways, indicating that these concepts will make it into mainstream languages soon. A fascinating opportunity for JIT-compiled languages like Java and C# is the possibility of selective code expansion at specific types. Instead of bloating parametric code by template-expanding it at compile-time, the same code can be selectively expanded at JIT compile-time, possibly taking into account runtime profile information to fine-tune performance.